MODULE 1

DATABASE MANAGEMENT SYSTEM

Databases and database technology have a major impact on the growing use of computers. It is fair to say that databases play a critical role in almost all areas where computers are used, including business, electronic commerce, engineering, medicine, genetics, law, education, and library science.

- Data
    - o Anything can be a data (raw facts and figures:- "Raw facts" refers to unprocessed and unorganized information, often in the form of figures, symbols, or observations, that are collected but not yet analyzed or interpreted)
- Information
    - o Processed Data (meaningful data)

Database Definition:

- A **database** is a collection of related data.
- For example, consider the names, telephone numbers, and addresses of the people you know. You may have recorded this data in database.
- **Defining** a database involves specifying the data types, structures, and constraints of the data to be stored in the database.

A **database management system (DBMS)** is a collection of programs that enables users to create and maintain a database. The DBMS is a general-purpose software(A general-purpose software system is designed to handle a wide range of tasks and is not limited to a specific function or industry) system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.

ADVANTAGES OF USING THE DBMS APPROACH

In this section we discuss some of the advantages of using a DBMS and the capabilities that a good DBMS should possess. The DBA(Database Administrator) must utilize these capabilities to accomplish a variety of objectives related to the design, administration, and use of a large multi-user database.

1. Controlling Redundancy

- In traditional software development utilizing file processing, every user group maintains its own files for handling its data-processing applications.
- For example, consider the UNIVERSITY database here, two groups of users might be the **course registration office** and the **accounting office.**
- In the traditional approach, each group independently keeps files on students.
- The accounting office keeps data related billing information, whereas the course registration office keeps track of student courses and grades.

This **redundancy (duplication)** in storing the same data multiple times leads to several problems.

In the database approach, the views of different user groups are integrated during database design.

Ideally, we should have a database design that stores each data item—such as a student's name or birth date—in ***only one place*** in the database. This is known as **data normalization**, and it ensures consistency and saves storage space. In this way we **controlled redundancy ,** which helps to improve the performance of queries.

2. Restricting Unauthorized Access

- When multiple users share a large database, it is likely that most users will not be authorized to access all information in the database.
- For example, financial data is often considered confidential, and only authorized persons are allowed to access such data. In addition, some users may only be permitted to **retrieve data**, whereas others are allowed to **retrieve and update**. Hence, the type of access operation— retrieval or update—must also be controlled.

- Typically, users or user groups are given account numbers protected by passwords, which they can use to gain access to the database. A DBMS should provide a **security and authorization subsystem**, which the DBA uses to create accounts and to specify account restrictions

3. Providing Persistent Storage for Program Objects

- **Databases** can be used to provide **persistent storage** for program objects and data structures. This is one of the main reasons for **object-oriented database systems**.
- Object-oriented database systems are compatible with programming languages such as C++ and Java, and the DBMS software automatically performs any necessary conversions.

4. Providing Storage Structures and Search- Techniques for Efficient Query Processing

- Database systems must provide capabilities for ***efficiently executing queries and updates.***

- Because the database is typically stored on disk, the DBMS must provide specialized data structures and search techniques to speed up disk search for the desired records.

- The **query processing and optimization** module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures.

5. Providing Backup and Recovery

- A DBMS must provide facilities for recovering from hardware or software failures.
- The **backup and recovery subsystem** of the DBMS is responsible for recovery.
- For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database **is restored to the state it was in before the transaction started executing.**

6. Providing Multiple User Interfaces

- Because many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces.
- These include **query languages** for casual users, **programming language** interfaces for application programmers, **forms and command codes** for parametric users, and **menu-driven interfaces and natural language interfaces** for standalone users.
- Both forms-style interfaces and menu-driven interfaces are commonly known as **graphical user interfaces (GUIs)**.

7. Representing Complex Relationships among Data

- A database may include numerous varieties of data that are interrelated in many ways.
- A DBMS must have the capability to represent a variety of complex relationships among the data, to define new relationships as they arise, and to retrieve and update related data easily and efficiently.

8. Enforcing Integrity Constraints

- Most database applications have certain **integrity constraints** that must hold for the data.
- A DBMS should provide capabilities for defining and enforcing these constraints. The simplest type of integrity constraint **involves specifying a data type for each data item**.
- It is the responsibility of the database designers to identify integrity constraints during database design.

9. Permitting Inferencing and Actions Using Rules

- In today's relational database systems, it is possible to associate **triggers** with tables. A trigger is a form of a rule activated by updates to the table, which results in performing some additional operations to some other tables, sending messages, and so on
- This functionality is provided by **active database systems**, which provide active rules that can automatically initiate actions when certain events and conditions occur.

CHARACTERISTICS OF THE DATABASE APPROACH

- **Self**-describing nature of a database system.
- Insulation between programs and data, and data abstraction.
- Support of multiple views of the data.
- Sharing of data and multiuser transaction processing.

Self-describing nature of a database system

- Database system is not only contain the data but also the complete definition and description of these data(structure)
- The information about database structure is called metadata
- Metadata is stored in the catalogue of the database

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
….	….	…..
….	….	…..
….	….	…..
Prerequisite_number	XXXXNNNN	PREREQUISITE

Figure 1.3
An example of a database catalog for the database in Figure 1.2.

Insulation between program and data

- In the file-based system, the structure of the data files is defined in the application programs so if a user wants to change the structure of a file, all the programs that access that file might need to be changed as well.

- On the other hand, in the database approach, the data structure is stored in the system catalogue and not in the programs.

- Therefore, one change is all that is needed to change the structure of a file.

Support for multiple views of data

- A database supports multiple views of data.
- A *view* is a subset of the database, which is defined and dedicated for particular users of the system.
- Multiple users in the system might have different views of the system. Each view might contain only the data of interest to a user or group of users.

Sharing of data and multiuser transaction processing.

- Current database systems are designed for multiple users.
- That is, they allow many users to access the same database at the same time.
- This access is achieved through features called *concurrency control strategies*. These strategies ensure that the data accessed are always correct and that data integrity is maintained.

DATABASE USERS

1. Database Administrators (DBA)

- This may be one person or a group of people in an organization responsible for authorizing access to the database, monitoring its use and managing all of the resources to support the use of the entire database system.
- DBA is the leader of the database. He is like a super user of the system
- DBA responsible for:

- ➢ Defining schema
- ➢ Defining instances
- ➢ Defining security
- ➢ Back-up and Recovery
- ➢ Monitoring the performances

2. System Analyst

- System analyst analyse the requirements of end users especially naïve-users.
- System analyst checks whether all the requirements of end users are satisfied.

3. Database Designers

- The database designer is responsible for designing the detailed database design, including tables, indexes, views, constraints etc.

- They controls what data to be stored and how the data items must be related.
- They communicate with system analyst

4. Application Programmers

- They are the developers who interact with the database by means of DML queries.
- These DML queries are written in the application programs like C, C++, JAVA, Pascal etc.These queries are converted into object code to communicate with the database.

5. End Users

End users are those who access the database from the terminal end

5.1 Naive Users

- ➢ Also called parametric users
- ➢ These are the users who use the existing application in daily basis to interact with the database

- For example, online library system, ticket booking systems, ATMs etc. which has existing application and users use them to interact with the database to fulfil their requests.

5.2 Casual users

- Also known as online users
- These are the **users** who occasionally access the **database** but they require different information each time.

5.3 Sophisticated Users

- They directly interact with the database by means of query language like SQL.
- These users will be scientists, engineers, analysts who thoroughly study SQL and DBMS to apply the concepts in their requirement.

5.4 Stand-alone Users

- These users will have stand –alone database for their personal use.
- These kinds of database will have readymade database packages which will have menus and graphical interfaces.
- Eg:User that maintains an address book

DATA MODELS, SCHEMAS, AND INSTANCES

A Data model- a collection of concepts that can be used to describe the structure of a database

Categories of data models

1. High-level or conceptual data models

2. Low-level or physical data models

3. Representational (or implementation) data models,

High-level or conceptual data models

- It provide concepts that are close to the way many users perceive data

- It uses entities, attributes and relationships
- Entity represent real world object (employee)
- Attribute represent some property that describes an entity (employee name)
- A relationship among two or more entities represents an association among two or more entities(relationship between an employee and a project)

Low-level or physical data models-

- It provides concepts that describe the details of how data is stored in the computer
- Physical data models describe how data is stored as files in the computer representing information such as record formats, record ordering and access paths.
- An access path is a structure that makes the search for particular database records efficient. An index is an example of an access path that allows direct access to data using an index term or a keyword.

Representational (or implementation) data models

- This level is in between high level and low level
- Which provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage.
- It hides many details of data storage on disk but can be implemented on a computer system directly.

Schemas, Instances, and Database State

- The description of a database is called the database schema.
- A displayed schema is called a schema diagram.
- The data in the database **at a particular moment in time** is called a database state or snapshot. it is also called the current set of occurrences or instances in the data-base.
- The distinction between data base schema and data base state is very important.

  ➢ When we define a new database, we specify its database schema only to the DBMS.

- The corresponding database state is the empty state with no data.
- We get the initial state of the database when the database is first populated or loaded with the initial data. From then on, every time an update operation is applied to the database we get another database state.

- At any point in time the database has a current state.
- The schema sometimes called the **intension**, and a database state is called an extension of the schema

Schema Diagram

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

THREE SCHEMA ARCHITECTURE

The goal of the three-schema architecture is to separate the user applications and the physical database. In this architecture, schemas can be defined at the following three levels.

1. The internal level

- It has an internal schema, which describes the physical storage structure of the database.
- The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.

2. The conceptual level

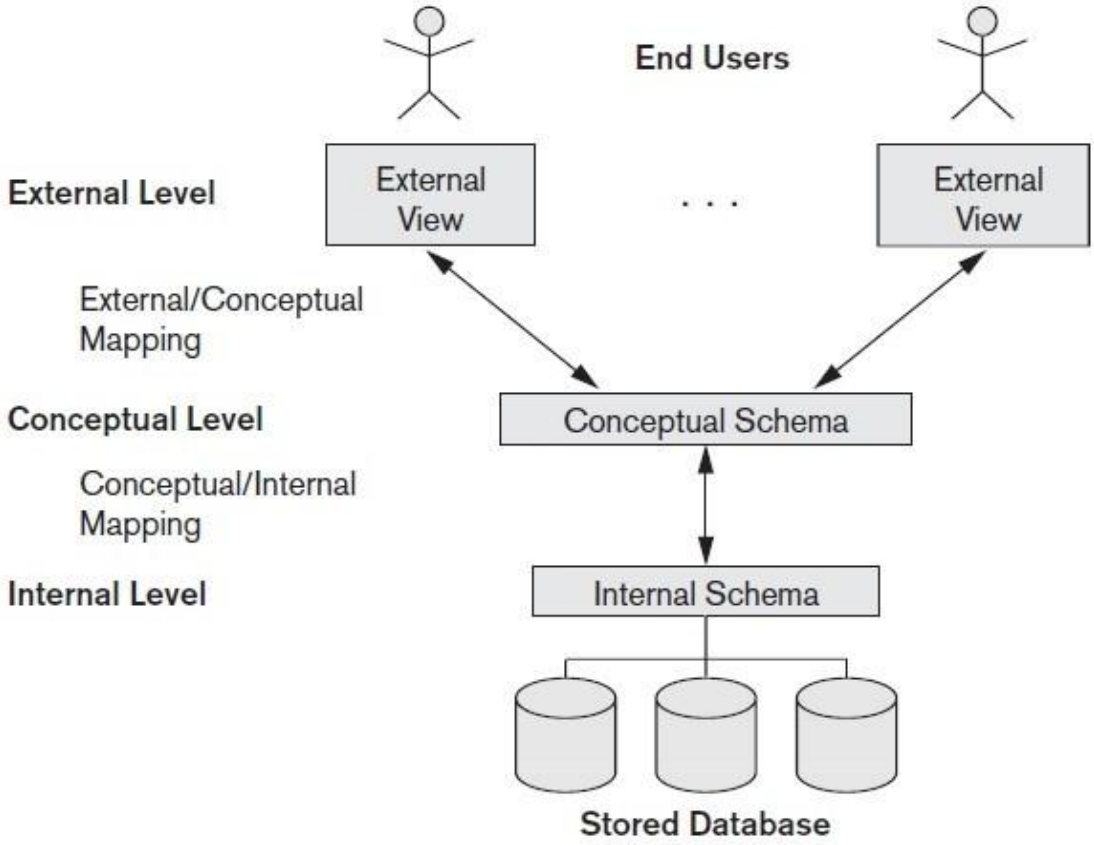
- ❖ It has a conceptual schema which describes the structure of the whole database for a community of users.
- ❖ The conceptual schema hides the details of physical storage structures and uses conceptual data model on describing entities, data type, relationships, user operations and constraints.

3. The external or view level

- ❖ It includes a number of external schemas or user views.(what user view)
- ❖ Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

The three-schema architecture is a convenient tool with which the user can visualize the schema levels in a database system. Most DBMSs do not separate the three levels completely and explicitly, but support the three-schema architecture to some extent.

The three-level architecture has an important place in database technology development because it clearly separates the users' external level, the database's conceptual level, and the internal storage level for designing a database.



DATA INDEPENDENCE

It can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level. We can define two types of data independence

1. logical data independence

- It is the capacity to change the conceptual schema without having to change external schemas or application programs.
- We may change the conceptual schema to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item).

- Changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.

2. Physical data independence

- It is the capacity to change the internal schema without having to change the conceptual schema.
- Hence, the external schemas need not be changed as well.
- Changes to the internal schema may be needed because some physical files were reorganized
- For example, by creating additional access structures—to improve the performance of retrieval or update. If the same data as before remains in the database, we should not have to change the conceptual schema.

The three-schema architecture can make it easier to achieve true data independence, both physical and logical

DBMS LANGUAGES

1. DDL (Data Definition Languages)

- It is used by the **DBA and by database designers** to define conceptual schema.
- In DBMS, DDL is also used to define internal and external schemas.

2. SDL(Storage Definition Language)

- In some DBMS, a separate language SDL is used to define the internal schema

3. VDL (View Definition Language)

- VDL is used to define external schemas

4.DML(Data Manipulation Language)

- Once the database schemas are compiled and the database is populated with data, users must have some means to manipulate the database.
- Typical manipulations include **retrieval, insertion, deletion and modification** of the data.
- The DBMS provides set of operations or a language called the data manipulation language.
- There are two main types of DMLs.

***A high-level or nonprocedural DML** can be used on its own to specify complex database operations. This can be embedded in a general-purpose programming language. For example SQL language

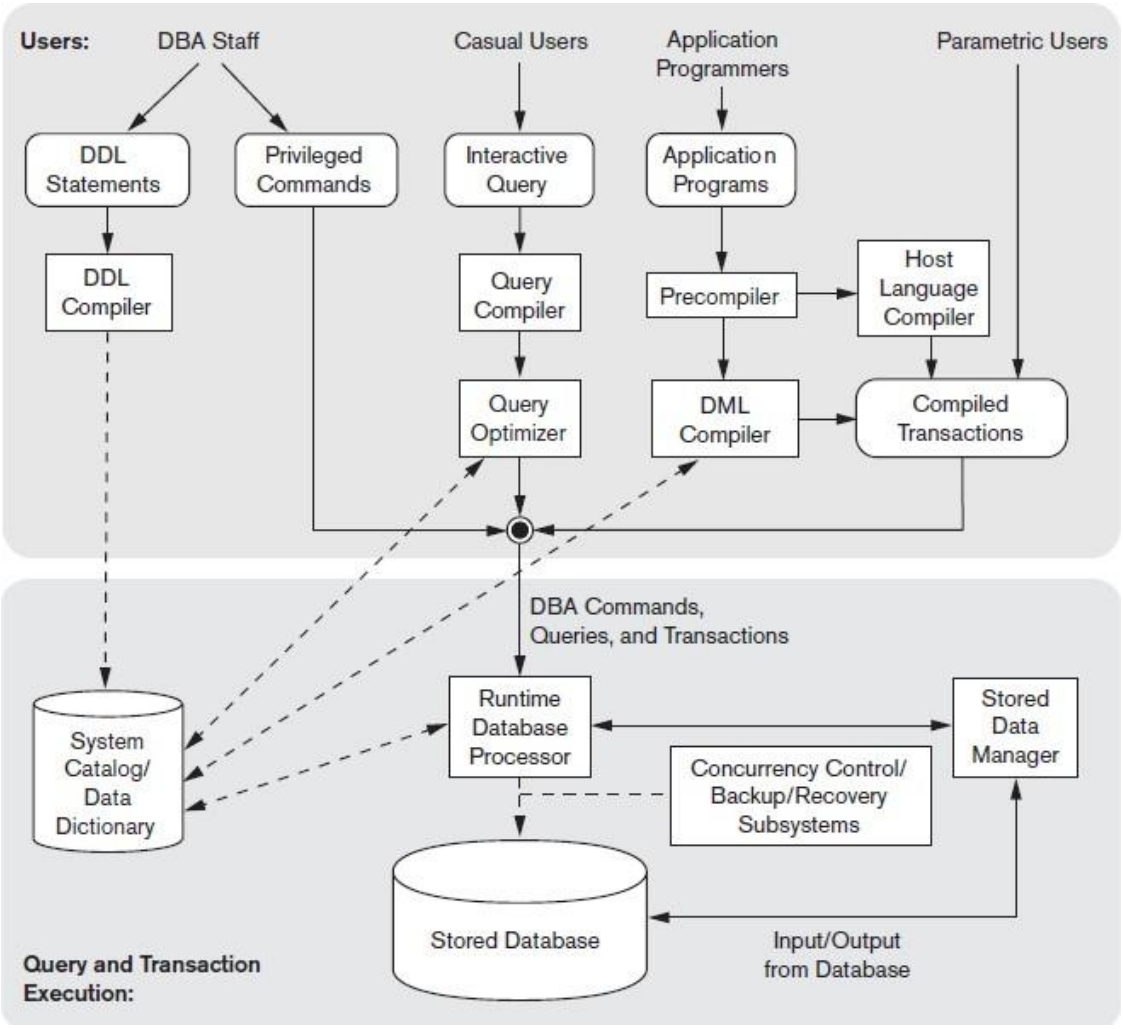
***A low-level or procedural DML** This type of DML retrieves records one record at a time. Therefore it needs to use programming language constructs, such as looping, to retrieve and process each record from set of a records.

Whenever DML commands, whether high level or low level are embedded in a general-purpose programming language, that language is called the host language and the DML is called the data sublanguage .

THE DATABASE SYSTEM ENVIRONMENT

DBMS Component Modules

A DBMS is a complex software system. In this section we discuss the types of software components that constitute a DBMS and the types of computer system software with which the DBMS interacts.



The figure is divided into two parts.

- The top part of the figure refers to the various users of the database environment and their interfaces.
- The lower part shows the internals of the DBMS responsible for storage of data and processing of transactions.

Top part

It shows interfaces for the

1. DBA staff : The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands.

2. casual users : who work with interactive interfaces to formulate queries.

3. application programmers : who create programs using some host programming languages.

4. parametric users : who do data entry work by supplying parameters to predefined transactions.

- The **DDL compiler** processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.

- **Casual users** and persons with occasional need for information from the database interact using some form of interface which we call the **interactive query** interface. **query compiler** that compiles the interactive query into an internal form. **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution.

- **Application programmers** write programs in **host languages** such as Java, C, or C++ that are submitted to a precompiler. The **precompiler** extracts DML commands from an application program written in a host programming language. These commands are sent to the **DML compiler** for compilation into object code for database access.

Lower part

- **runtime database processor** executes (1) the privileged commands, (2) the executable query plans, and (3) the canned transactions with runtime parameters.

- It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory